SQL_TEST_01 · BUILD 3.5 · HANDOFF BRIEF

Documentation RAG Agent

A standalone, deep-dive reference for Build 3.5 — every phase, every file, every verification result, and every real gap found along the way, in full. Complements (does not replace) the living cheat sheet and the Phase 1 flow diagram.

STATUS	COMMITS	CONTINUES
Complete — all 3 phases	`d3fb77f` → `f12b732`	Build 3 (retrieval-augmented agent)

PRECEDES

Build 4 (observability & eval harness)

What Was Built, and Why

Build 3 gave the agent semantic search over `appdb`'s own data. Build 3.5 applies that exact same pattern — pgvector, sentence-transformers, a distance threshold — to a genuinely different corpus: this project's own documentation, so the agent can answer questions about its own build history and design decisions. Reusing the pattern rather than inventing a new one is deliberate, per this roadmap's own "compound, don't sprawl" principle — but the underlying problem (chunking prose instead of embedding single-row summaries) turned out to be a real, distinct challenge, not a repeat of Build 3.

Chunking strategy decided before building: per-callout/code-block, not per-section (too uneven in length) and not fixed-size windows (risks splitting a decision from its own rationale). These docs were already written in small, self-contained decision/rationale units — reusing that existing structure rather than inventing a new one.

1 Chunk & Embed the Docs

Goal: extract chunks from the project's own documentation, embed them, and prove raw similarity search works standalone — before touching the agent.

```
010_docs_chunks_schema.sql
```

```sql
CREATE SCHEMA IF NOT EXISTS docs;

CREATE TABLE IF NOT EXISTS docs.chunks (
    chunk_id     SERIAL PRIMARY KEY,
    source_file  TEXT NOT NULL,
    chunk_text   TEXT NOT NULL,
    embedding    vector(384)
);

GRANT USAGE ON SCHEMA docs TO appdb_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA docs TO appdb_reader;
ALTER DEFAULT PRIVILEGES IN SCHEMA docs GRANT SELECT ON TABLES TO appdb_reader;
```

`embedding` is nullable on purpose — mirrors Build 3's split between adding a column and backfilling it. Grants go to `appdb_reader` only: this table is populated by an offline script running as the admin user (like `embed_summaries.py`), never written to by the agent at runtime.

Four extraction shapes, because the corpus actually has four — this project's documentation spans two design eras:

```
extract_doc_chunks.py — the four extractors
```

```python
def extract_callout_chunks(soup):      # .callout divs + pre.code blocks — Build 2/2.5/3 briefs
def extract_table_row_chunks(soup):    # table.ref rows — the cheat sheet
def extract_finding_chunks(soup):      # .finding/.risk/.vlog/.continuity — pre-Build-1 + Build 1 briefs
def extract_narrative_chunks(soup):    # .lede/.doc-subtitle/.section-note/.phase-goal — added after a real gap, below
```

Real gap found — Build 1's two handoff briefs existed only as published Artifacts, never as local files. They couldn't be chunked because there was nothing on disk to parse. Fixed by fetching both via `WebFetch` — which, for `claude.ai/code/artifact/<uuid>` URLs the user owns, returns full raw HTML rather than the usual markdown conversion — and saving clean local copies in `PROJECT_DIAGRAMS/BUILD_1_FLOW/` (`build1-`

handoff-brief.html, environment-handoff-brief-v2.html). Those two briefs use an older class scheme (.finding/.risk/.vlog/.continuity) that predates .callout/pre.code — hence the fourth extractor.

embed_chunks.py

```
with conn.cursor() as cur:
    cur.execute("SELECT chunk_id, chunk_text FROM docs.chunks WHERE embedding IS NULL")
    rows = cur.fetchall()

if not rows:
    print("No chunks need embedding - everything is already up to date.")
else:
    embeddings = model.encode([text for _, text in rows], show_progress_bar=True)
    # UPDATE ... WHERE chunk_id = %s, same idempotent pattern as Build 3's embed_summaries.py fix
```

The overview chunk failed twice, for two different reasons — worth walking through both. A project-level "what is this" answer doesn't exist anywhere in the corpus by default; every brief's .lede is scoped to its own build. **First attempt:** one dense ~180-word paragraph covering all five builds, added as PROJECT_DIAGRAMS/project-overview.html. Direct measurement (embedding <=> query, checked in isolation, not assumed) showed it scored **0.8247** distance against "What is this project about?" — worse than 16 other chunks, including generic ones like "a reference for Build X." Root cause: all-MiniLM-L6-v2 embeds long, multi-topic text as an average across topics, which stops matching short, generic queries well — chunk length/style needs to roughly match expected query length/style, not just be topically relevant. **Fix:** rewrote it as nine short, single-idea lines instead of one paragraph. The best-matching line then scored **0.6536** and ranked first.

Verified live: 230 chunks across 6 source files, all embedded at 384 dimensions. A narrow question ("why does agent_scratch have two roles?") scored 0.34-0.43 distance with precise, on-topic results — proving raw similarity search genuinely works before any agent wiring.

2 Wire Into the Agent

Goal: expose the docs corpus as a real tool, and give the agent enough judgment to pick the right retrieval strategy per question type.

```
tools.py — search_docs (new function)

DOCS_SIMILARITY_DISTANCE_THRESHOLD = 0.70  # provisional - not calibrated with Build 3's 40-query rigor; revisit if too loose/strict

def search_docs(query_text: str, limit: int = 5) -> list[dict]:
    model = _get_embedding_model()  # shared cache, same model instance search_summaries already loaded
    query_embedding = model.encode(query_text)

    conn = psycopg.connect(
        host=os.environ.get("POSTGRES_HOST", "127.0.0.1"),
        port=5432,
        dbname=os.environ["POSTGRES_DB"],
        user=os.environ["POSTGRES_READER_USER"],
        password=os.environ["POSTGRES_READER_PASSWORD"],
    )
    register_vector(conn)
    with conn.cursor() as cur:
        cur.execute("""
            SELECT source_file, chunk_text, embedding <=> %s AS distance
            FROM docs.chunks
            WHERE embedding <=> %s < %s
            ORDER BY distance
            LIMIT %s
        """, (query_embedding, query_embedding, DOCS_SIMILARITY_DISTANCE_THRESHOLD, limit))
        rows = cur.fetchall()
    conn.close()

    if not rows:
        return [{"message": f"No sufficiently relevant documentation found for {query_text!r} — nothing scored below the relevance threshold."}]

    return [{"source_file": sf, "chunk_text": ct, "distance": round(float(d), 4)} for sf, ct, d in rows]
```

Why this shape, exactly: deliberately identical to `search_summaries` — same threshold-cutoff pattern, same "message instead of empty list" behavior when nothing clears the bar, same connection-as-`appdb_reader` convention. No reason to invent a second retrieval shape when the first one already

works and the corpus difference (allocations vs. doc chunks) doesn't change how similarity search itself needs to behave.

```
tools.py — list_tables / describe_table (widened)

-- before:
WHERE table_schema IN ('public', 'clean', 'agent_scratch')
-- after, in both functions:
WHERE table_schema IN ('public', 'clean', 'agent_scratch', 'docs')
```

Why this was necessary at all: the `GRANT SELECT` on `docs.chunks` to `appdb_reader` already existed from Phase 1's migration — the role could technically query it. But `list_tables`/`describe_table` are the agent's *only* way of discovering what exists; a grant the discovery query never surfaces is invisible to the model regardless of what it's technically permitted to touch. Same pattern every prior build has followed (Build 2 added `clean`, Build 2.5 added `agent_scratch`) — permission and discoverability are two separate things, and both have to be extended together.

```
agent.py — new tool schema

{
    "name": "search_docs",
    "description": "Semantic search over this project's own documentation (handoff briefs, cheat
sheet, project overview) - build history, design decisions, and rationale. Use for thematic or
'why'/'how' questions about the project itself, not the appdb database. Returns a message, not an
error, if nothing is sufficiently relevant. Note: returns individual similar chunks, not an
aggregated list - for 'list all X' questions (e.g. every file, every SQL migration), run_sql_query
directly against docs.chunks works better than semantic search.",
    "input_schema": {
        "type": "object",
        "properties": {
            "query_text": {
                "type": "string",
                "description": "A natural-language question about the project's history, design
decisions, or documentation."
            }
        },
        "required": ["query_text"]
    }
}
```

The tool description itself carries the enumeration-vs-semantic guidance, not just the system prompt. Tool descriptions are visible to the model at every turn it considers calling that tool, whereas the system prompt is one

message among many by the time a multi-turn conversation runs long. Putting the "use run_sql_query instead for 'list all X'" guidance in both places was deliberate belt-and-suspenders, not redundant.

```
agent.py — dispatch addition

elif block.name == "search_docs":
    result = search_docs(block.input["query_text"])
```

```
agent.py — SYSTEM_PROMPT addition

For questions about this project's own history, design decisions, or documentation,
use search_docs; for "list every X" style questions about the documentation itself,
prefer run_sql_query against docs.chunks instead.
```

Real gap found via live agent testing — top-k similarity search cannot answer "list all X" questions, and no embedding model fixes that. Enumeration ("list the files used in this project") is fundamentally an aggregation task, not a similarity-matching task — no single chunk (or top-5 set) constitutes "the complete list," so distance-based retrieval structurally cannot satisfy it regardless of embedding quality. Fixed via the tool-description and system-prompt guidance above, steering the agent toward `run_sql_query` directly against `docs.chunks` for enumeration-style questions instead — the same combine-tools pattern Build 3 already proved (`search_summaries` plus a follow-up SQL query for exact dates). Full design discussion below.

Verified live — asked `"Why does agent_scratch have two separate database roles?"`. The model called `search_docs`, judged its own first query too broad, called it again with a more specific phrase, then answered correctly — directly quoting the real "why two roles, not one wider role" rationale from Build 2.5's own brief. `stop_reason: end_turn`, no errors, no missing-dispatch issues.

Design Discussion — Would a Different Model Have Helped?

Raised directly during this phase: "not sure if we're happy with the tool. Would it be best to use another LLM for more meaningful results?" Worth resolving explicitly, since "another LLM" conflates two genuinely different models people often lump together.

OPTION	PRO	CON
Bigger local embedding model (e.g. `all-mpnet-base-v2`)	Still free; generally better retrieval quality than MiniLM	Slower, larger; no evidence model quality (vs. chunk content) was the actual bottleneck

OPTION	PRO	CON
Paid embedding API (Voyage AI / OpenAI `text-embedding-3`)	Likely higher quality still	Real per-call cost; explicitly declined for the same reason back in Build 3
Keep the same embedding model, fix chunk content/length instead	Directly addresses the actual root causes found (chunk-length/style mismatch, then task-type mismatch); free, no new dependency	Iterative — took two rounds of real, measured testing to land on
Swap the generative LLM (the model answering, not retrieving)	Could improve answer synthesis quality once retrieval is already good	Doesn't touch retrieval at all — every "results feel off" test up to this point was `search_chunks.py`, a standalone script with zero generative LLM in the loop

Decision: no model swap, either layer. Diagnosis showed both real gaps found this build had specific, measurable, fixable root causes unrelated to model quality — the overview chunk's length/style mismatch (Phase 1, confirmed via direct `embedding <=> query` distance measurement) and the structural fact that top-k similarity search cannot enumerate (Phase 2, a task-type mismatch no embedding model changes). Fixing the actual causes resolved what testing could measure; swapping models would have added cost and complexity while leaving both root causes completely untouched.

3 Automated Tests

Goal: codify the manual verification above as a real pytest suite, mirroring `test_semantic_search.py`'s exact shape.

test_docs_search.py

```
def test_all_chunks_have_embeddings_with_correct_dimensions(): ...
def test_relevant_query_returns_real_results(): ...
def test_irrelevant_query_returns_no_match_message(): ...
def test_threshold_regression_guard(): ...
def test_docs_schema_discoverable_via_list_tables(): ...
```

> **The "irrelevant query" test case was verified empirically before being hardcoded** — `"best chocolate chip cookie recipe"` was checked directly against `search_docs` first (confirmed it returns the no-match message) rather than assumed, avoiding a flaky test built on a guess.

Result: all 5 new tests passing on first run. Full project suite: **19/19 passing** (14 prior across `test_data_quality.py`, `test_agent_scratch_boundary.py`, `test_semantic_search.py`, plus these 5).

Verification Log

What was actually exercised against the running system, not just read for plausibility — same standard as every build in this project.

- **Extraction across all 6 source files**
  230 chunks total, spanning both the current (`.callout`/`pre.code`) and pre-Build-1 (`.finding`/`.risk`) design eras.
- **Embedding coverage**
  230/230 chunks embedded, all at 384 dimensions, verified with an unlimited (not sampled) query after a command typo accidentally removed the intended `LIMIT`.
- **Narrow-query relevance**
  `"why does agent_scratch have two roles?"` — distances 0.25-0.37 after the final threshold/agent wiring, precise on-topic chunks.
- **Overview-chunk fix, measured directly**
  Dense paragraph: 0.8247 distance, unranked in top 16. Nine short lines: best line at 0.6536, ranked first. Confirmed via isolated `embedding <=> query` distance checks, not inferred from top-5 output alone.
- **Enumeration-question gap, confirmed real**
  "List the files used in this project" scored worse and returned tangential chunks even post-fix — a genuine structural limit of similarity search, not an embedding-quality problem; addressed via agent tool-choice guidance instead of further retrieval tuning.
- **Live agent run, full round trip**
  Model self-corrected across two `search_docs` calls, then answered correctly with a direct, accurate quote from Build 2.5's brief.
- **Full automated suite**
  19/19 passing — 5 new in `test_docs_search.py`, alongside all 14 prior tests.

Known Limitations & Open Items

`DOCS_SIMILARITY_DISTANCE_THRESHOLD = 0.70` is provisional, not rigorously calibrated. Build 3's `SIMILARITY_DISTANCE_THRESHOLD` came from a 40-query statistical calibration (20 in-domain, 20 out-of-domain). This threshold is instead grounded in the handful of real distances observed during this session's own testing — a reasonable first pass, but a proper `calibrate_docs_threshold.py` pass would be a legitimate follow-up if this proves too loose or too strict in practice.

Enumeration/"list all X" questions remain a structural limitation, not a solved problem. The agent can route around it via `run_sql_query` against `docs.chunks`, but that depends on the model's own judgment to choose the right tool each time — there's no hard guarantee it always will. Worth watching if this pattern gets exercised more heavily later.

Build 1's two handoff briefs are now local copies, not a live sync with their source Artifacts. If those Artifacts are ever edited again independently, the local copies (and therefore the chunked corpus) would silently go stale — no automated re-fetch exists.

Not done, deliberately — true hierarchical RAG (per-document summary chunks). Flagged during this build's design discussion as a future-implementation item, not urgent: the single project-overview chunk turned out sufficient once its length/style was fixed, so a full per-document-summary layer isn't currently justified. Revisit if the doc corpus grows meaningfully past its current 6 files.

Environment State

COMPONENT	DETAIL
New schema	`docs`
New SQL file	`010_docs_chunks_schema.sql`
New Python	`extract_doc_chunks.py, embed_chunks.py, search_chunks.py, test_docs_search.py`
New/changed Python	`tools.py (+search_docs, widened discovery), agent.py (+tool, +dispatch, +prompt line)`
New dependency	`beautifulsoup4`
New local files	`PROJECT_DIAGRAMS/BUILD_1_FLOW/build1-handoff-brief.html, environment-handoff-brief-v2.html, PROJECT_DIAGRAMS/project-overview.html`
Corpus size	`230 chunks across 6 source files, all embedded at 384 dimensions`
Role used	`appdb_reader (no new role needed for this build)`

Build 3.5 complete. Next: Build 4 — agent observability & eval harness (JSON Lines tool-call logging decided as the Phase 1 approach). Ongoing cross-build command reference lives in `sql-test-01-cheatsheet.html`.